Movie Recommendation System Project Report

A structured report on a content-based movie recommender built with Streamlit, cosine similarity, and TMDB poster integration.

Project type	Machine learning / recommendation system
Interface	Streamlit web application
Core method	Cosine similarity over precomputed movie vectors
Output	Top 10 similar movies with posters
External service	TMDB poster API

Purpose: help users quickly discover movies similar to a chosen title, while presenting results in an interactive and visually clear interface.

Prepared from the project files and implementation.

1. Problem Statement

Users often spend excessive time browsing large movie catalogs. A recommendation system reduces this search effort by ranking titles that are most similar to a selected movie.

This project addresses the problem by building a content-based movie recommender that uses movie features and cosine similarity to return the most relevant alternatives.

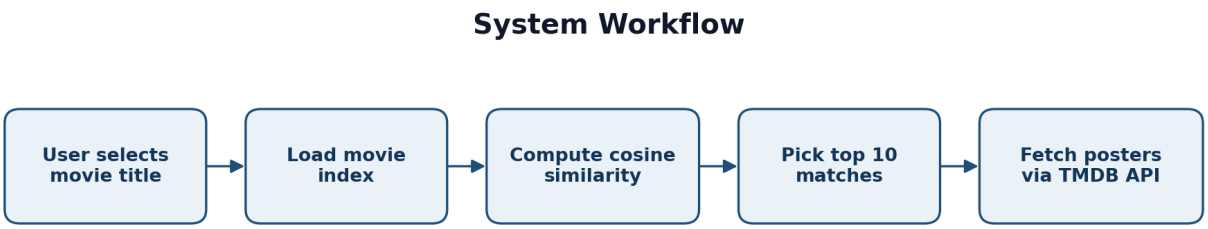
2. Project Objectives

- Accept a movie title from the user through a simple Streamlit interface.
- Identify the most similar movies using a precomputed similarity matrix.
- Show the top 10 recommendations in a clean grid layout.
- Fetch posters dynamically so the result is visually informative.

3. Project Snapshot

Aspect	Details
Model type	Content-based recommender
Similarity metric	Cosine similarity
UI framework	Streamlit
Data handling	Pandas + Pickle
API integration	TMDB movie poster endpoint
Display format	2 rows × 5 posters

4. System Workflow



Content-based recommendation pipeline used in the Streamlit application

The application first takes a user-selected movie title, then searches for the corresponding row in the dataset, computes similarity scores, selects the best matches, and finally retrieves posters for display.

5. Implementation Overview

Core logic from the application code

- **Data loading:** the app opens *movie_data.pkl* with pickle to load the movie dataset and cosine similarity matrix.
- **Recommendation function:** the selected title is matched to its index, similarity scores are sorted in descending order, and the top 10 indices are returned.
- **Poster fetching:** the movie ID is sent to TMDB's movie details endpoint and the returned poster path is converted into a full image URL.
- **Interface:** Streamlit provides the title, dropdown, button, and poster grid.

Technology stack

Component	Role in the project
Python	Primary implementation language
Pandas	Dataset handling and indexing
Pickle	Loading serialized movie data and similarity matrix
Requests	Calling the TMDB API
Streamlit	User interface and output rendering
Cosine similarity	Ranking similar movies

6. Results and Output Preview

The interface displays the top 10 recommended movies as poster cards. This makes the output easier to scan than a text-only list and improves the overall presentation of the recommender.

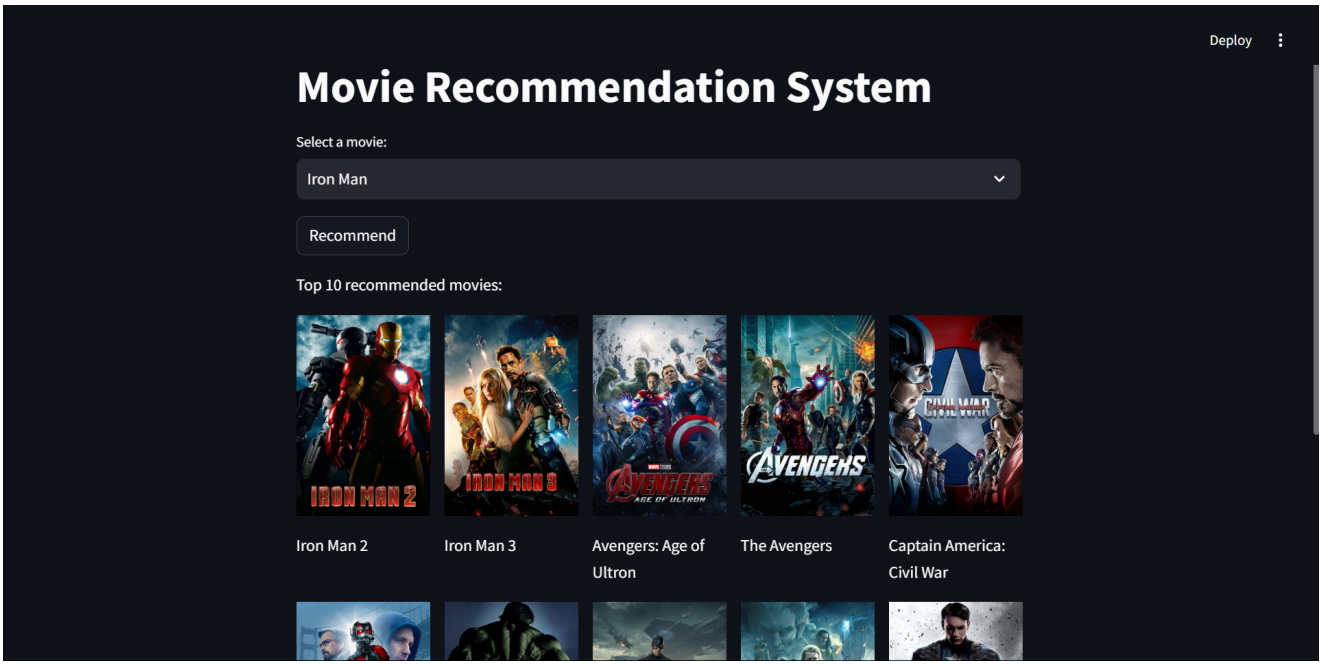


Figure: Sample Streamlit output showing recommendations for Iron Man.

7. Challenges and Observations

Challenge	Observation / practical response
Large processed files	The project relies on a preprocessed pickle file, which keeps the app fast at runtime.
API key handling	The poster API needs secure key management before public deployment.
Poster availability	Some movies may return missing or incomplete poster data from the API.
Layout density	A poster grid is more readable than a plain text recommendation list.

8. Future Improvements

- Add collaborative filtering or a hybrid recommendation approach.
- Use environment variables for secure API key storage.
- Add caching to reduce repeated poster API calls.
- Include search, filters, and user-specific preferences.
- Deploy the app online for easy access.

9. Conclusion

This project is a clear example of how machine learning ideas can be used in a simple and useful web application. It combines recommendation logic, data processing, API integration, and UI design to create a practical movie discovery tool.

The final result is an interactive system that returns similar movies quickly and presents them in a visually appealing layout.

End of report